100. Возврат значений по ссылке, по адресу и по значению

На трех предыдущих уроках мы узнали о передаче аргументов в функции **по значению**, **по ссылке** и **по адресу**. На этом уроке мы рассмотрим возврат значений обратно из функции в вызывающий объект всеми этими тремя способами.

Возврат значений (с помощью оператора return) работает почти так же, как и передача значений в функцию. Все те же плюсы и минусы. Основное отличие состоит в том, что поток данных двигается уже в противоположную сторону. Однако здесь есть еще один нюанс — **локальные переменные**, которые выходят из **области видимости** и уничтожаются, когда функция завершает свое выполнение.

Оглавление:

1. **Возврат по значению**
2. **Возврат по адресу**
3. **Возврат по ссылке**
4. **Смешивание возвращаемых значений и ссылок**
5. **Заключение**
6. **Тест**

Возврат по значению

Возврат по значению — это самый простой и безопасный тип возврата. При возврате по значению, копия возвращаемого значения передается обратно в caller. Как и в случае с передачей по значению, вы можете возвращать **литералы** (например, `7`), переменные (например, `x`) или выражения (например, `x + 2`), что делает этот способ очень гибким.

Еще одним преимуществом является то, что вы можете возвращать переменные (или выражения), в вычисление которых задействованы и локальные переменные, объявленные в теле самой функции. При этом, можно не беспокоиться о проблемах, которые могут возникнуть с областью видимости. Поскольку переменные вычисляются до того, как функция производит возврат значения, то здесь не должно быть никаких проблем с областью видимости этих переменных, когда заканчивается блок, в котором они объявлены. Например:

```
int doubleValue(int a)
{
    int value = a * 5;
    return value; // value возвращается здесь
} // value уничтожается здесь
```

Возврат по значению идеально подходит для возврата переменных, которые были объявлены внутри функции, или для возврата значений функции, которые были переданы по значению. Однако, подобно передаче по значению, возврат по значению медленный при работе со **структурами** и классами.

Когда использовать возврат по значению:

- при возврате переменных, которые были объявлены внутри функции;
- при возврате аргументов функции, которые были переданы в функцию по значению.

Когда не использовать возврат по значению:

- при возврате **стандартных массивов** или **указателей** (используйте возврат по адресу);
- при возврате больших структур или классов (используйте возврат по ссылке).

Возврат по адресу

Возврат по адресу — это возврат адреса переменной обратно в caller. Подобно передаче по адресу, возврат по адресу может возвращать только адрес переменной. Литералы и выражения возвращать нельзя, так как они не имеют адресов. Поскольку при возврате по адресу просто копируется адрес из функции в caller, то этот процесс также очень быстрый.

Тем не менее, этот способ имеет один недостаток, который отсутствует при возврате по значению: если вы попытаетесь возвратить адрес локальной переменной, то получите неожиданные результаты. Например:

```
int* doubleValue(int a)
{
    int value = a * 5;
    return &value; // value возвращается по адресу здесь
} // value уничтожается здесь
```

Как вы можете видеть, `value` уничтожается сразу после того, как ей адрес возвращается в caller. Конечным результатом будет то, что caller получит адрес освобожденной памяти (**висячий указатель**), что, несомненно, вызовет проблемы. Это одна из самых распространенных ошибок, которую делают новички. Большинство современных компиляторов выдадут предупреждение (а не ошибку), если программист попытается вернуть локальную переменную по адресу. Однако есть несколько способов обмануть компилятор, чтобы сделать что-то «плохое», не генерируя при этом предупреждения, поэтому вся ответственность лежит на программисте, который должен гарантировать, что возвращаемый адрес будет корректен.

Возврат по адресу часто используется для возврата динамически выделенной памяти обратно в caller:

```
int* allocateArray(int size)
{
    return new int[size];
}

int main()
{
    int *array = allocateArray(20);

    // Делаем что-нибудь с array

    delete[] array;
    return 0;
}
```

Здесь не возникнет никаких проблем, так как динамически выделенная память не выходит из области видимости в конце блока, в котором объявлена, и все еще будет существовать, когда адрес будет возвращаться в caller.

Когда использовать возврат по адресу:

- при возврате динамически выделенной памяти;
- при возврате аргументов функции, которые были переданы по адресу.

Когда не использовать возврат по адресу:

- при возврате переменных, которые были объявлены внутри функции (используйте возврат по значению);
- при возврате большой структуры или класса, который был передан по ссылке (используйте возврат по ссылке).

Возврат по ссылке

Подобно передаче по ссылке, значения, возвращаемые по ссылке, должны быть переменными (вы не сможете вернуть ссылку на литерал или выражение). При **возврате по ссылке** в caller передается ссылка на переменную. Затем caller может её использовать для продолжения изменения переменной, что может быть иногда полезно. Этот способ также очень быстрый и при возврате больших структур или классов.

Однако, как и при возврате по адресу, вы не должны возвращать локальные переменные по ссылке. Рассмотрим следующий фрагмент кода:

```
int& doubleValue(int a)
{
    int value = a * 5;
    return value; // value возвращается по ссылке здесь
} // value уничтожается здесь
```

В программе, приведенной выше, возвращается ссылка на переменную `value`, которая уничтожится, когда функция завершит свое выполнение. Это означает, что caller получит ссылку на мусор. К счастью, ваш компилятор, вероятнее всего, выдаст предупреждение или ошибку, если вы попытаетесь это сделать.

Возврат по ссылке обычно используется для возврата аргументов, переданных в функцию по ссылке. В следующем примере мы возвращаем (по ссылке) элемент массива, который был передан в функцию по ссылке:

```
#include <iostream>
#include <array>

// Функция getElement() для поиска элемента по индексу index
int& getElement(std::array<int, 20> &array, int index)
{
    // Мы знаем, что array[index] не уничтожится, когда мы будем возвращать данные в caller (так как caller сам передал нам array)
    // Так что здесь не должно быть никаких проблем с возвратом по ссылке
    return array[index];
}

int main()
{
    std::array<int, 20> array;

    // Присваиваем элементу массива под индексом 15 значение 7
    getElement(array, 15) = 7;

    std::cout << array[15] << '\n';

    return 0;
}
```

Результат выполнения программы:

```
7
```

Когда мы вызываем `getElement(array, 15)`, то getElement() возвращает ссылку на элемент массива под индексом 15, а затем main() использует эту ссылку для присваивания этому элементу значения `7`.

Хотя этот пример непрактичен, так как мы можем напрямую обратиться к 15 элементу массива, но как только мы будем рассматривать классы, то вы обнаружите гораздо больше применений для возврата значений по ссылке.

Когда использовать возврат по ссылке:

- при возврате ссылки-параметра;
- при возврате элемента массива, который был передан в функцию;
- при возврате большой структуры или класса, который не уничтожается в конце функции (например, тот, который был передан в функцию).

Когда не использовать возврат по ссылке:

- при возврате переменных, которые были объявлены внутри функции (используйте возврат по значению);
- при возврате стандартного массива или значения указателя (используйте возврат по адресу).

Смешивание возвращаемых значений и ссылок

Хотя функция может возвращать как значение, так и ссылку, caller может неправильно это интерпретировать. Посмотрим, что произойдет при смешивании возвращаемых значений и ссылок на значения:

```
int returnByValue()
{
    return 7;
}

int& returnByReference()
{
    static int y = 7; // static гарантирует ну, что переменная y не уничтожится, когда выйдет из локальной области видимости
    return y;
}

int main()
{
    int value = returnByReference(); // случай A: всё хорошо, обрабатывается как возврат по значению
    int &ref = returnByValue(); // случай B: ошибка компилятора, так как 7 - это r-value, а r-value не может быть привязано к
    const int &cref = returnByValue(); // случай C: всё хорошо, время жизненного значения продлевается в соответствии
}
```

В случае A мы присваиваем ссылку возвращаемого значения переменной, которая сама не является ссылкой. Поскольку `value` не является ссылкой, то возвращаемое значение просто копируется в `value` так, как если бы returnByReference() был возвратом по значению.

В случае B мы пытаемся инициализировать ссылку `ref` копией возвращаемого значения функции returnByValue(). Однако, поскольку возвращаемое значение не имеет адреса (это **r-value**), мы получим ошибку компиляции.

В случае C мы пытаемся инициализировать **константную ссылку** `cref` копией возвращаемого значения функции returnByValue(). Поскольку константные ссылки могут быть инициализированы с помощью r-values, то здесь не должно быть никаких проблем. Обычно r-values уничтожаются в конце выражения, в котором они созданы, однако, при привязке к константной ссылке, время жизни r-value (в данном случае, возвращаемого значения функции) продлевается в соответствии со временем жизни ссылки (в данном случае, `cref`).

Заключение

В большинстве случаев идеальным вариантом для использования является возврат по значению. Это также самый гибкий и безопасный способ возврата данных обратно в вызывающий объект. Однако возврат по ссылке или по адресу также может быть полезен при работе с динамически выделенной памятью. При использовании возврата по ссылке или по адресу убедитесь, что вы не возвращаете ссылку или адрес локальной переменной, которая выйдет из области видимости, когда функция завершит свое выполнение!

Тест

Напишите **прототипы** для каждой из следующих функций. Используйте наиболее подходящие параметры и типы возврата (по значению, по адресу или по ссылке). Используйте const, когда это необходимо.

Задание №1

Функция sumTo(), которая принимает целочисленный параметр, а возвращает сумму всех чисел между `1` и числом, которое ввел пользователь.

Задание №2

Функция printAnimalName(), которая принимает структуру `Animal` в качестве параметра.

Задание №3

Функция minmax(), которая принимает два целых числа в качестве входных данных, а возвращает наименьшее и наибольшее числа в качестве отдельных параметров.

> ***Подсказка:*** Используйте параметры вывода.

Задание №4

Функция getIndexOfLargestValue(), которая принимает целочисленный массив (как указатель) и его размер, а возвращает индекс наибольшего элемента массива.

Задание №5

Функция getElement(), которая принимает целочисленный массив (как указатель) и индекс и возвращает элемент массива по этому индексу (не копию элемента). Предполагается, что индекс корректен, а возвращаемое значение — константное.

Ответ №1

```
int sumTo(const int value);
```

Ответ №2

```
void printAnimalName(const Animal &animal);
```

Ответ №3

```
void minmax(const int a, const int b, int &minOut, int &maxOut);
```

Ответ №4

```
int getIndexOfLargestValue(const int *array, const int length);
```

Ответ №5

```
const int& getElement(const int *array, const int index);
```